TIVIDY

A Generic Workflow Management System

The Demo Script

Written by

The Power Rangers

Rayhaan Ahmed

Tashil Haripersad

Kyle Naicker

Haytham Phillips

Oratile Lesedi Rabotapi

INTRO - 1 MIN

TASHIL

When you hear the word ‘generic’, what’s the first thing that pops into your head? A no-name brand cereal? Templatised C++ functions? Or a blank document template?

For us, wrapping our heads around building a ‘Generic’ Workflow Management System was... a process. Because instead of hard-coding just one specific set of tasks, we had to build an engine that could run virtually any process.

Prof. Marshall always says, ‘go play with it’. And so, we did. A lot. In fact, we went through three entirely different versions of our design before finally landing… [*DRUMROLL]* on this one.

RAYHAAN

Ain’t she a beaut?

TASHIL

Good morning, everyone. We’re the Power Rangers, and our team is proud to present TIVIDY—our attempt at a truly Generic Workflow Management System.

OUR DESIGN:

BUILDER & PROTOTYPE – 1 MIN

ORATILE

Let’s start with how a workflow gets built. A workflow can have nested groups, individual work items, and extra behaviour, so building one by hand gets messy really quickly.

The Director tells the Builder which pieces to add and in what order. However, the client still has access to the Builder, allowing them to create their own custom workflows.

Once a WorkflowDefinition is constructed, we don’t want to rebuild that complex object from scratch every time. So, we use Prototype to clone it. We’re performing a deep copy, so working on one instance won't touch the definition.

COMPOSITE, MEDIATOR, STATE & STRATEGY - 2 MIN

TASHIL

Composite is how we represent the workflow’s structure. A WorkItem is one task, while a WorkGroup is a step that can contain other items or groups. Whatever sits inside a group runs in parallel, and the top-level steps run sequentially.

But who decides when the next component is released? The Mediator. If we look at the Sequence Diagram, once WI2 finishes, it notifies the Mediator. The Mediator then checks WG1; if WI1 is still busy, it does nothing. If WI1 is complete, the group is complete, so the Mediator releases WI3.

A WorkItem owns its State, which is used to decide which operations are legal. An interesting transition: reject something from Approval, and it can be restarted. Anywhere else, rejection is final.

Of course, work needs to be done by someone. We use Strategy to define a family of assignment algorithms. RoleAssignment filters by role, LeastLoadedAssignment sorts by workload, and RuleAssignment picks at random. We start with the workflow's entire participant pool and execute our strategies in order, each narrowing the last result. No strategies passed in? Well, our system also accommodates for manual assignment.

DECORATOR, CHAIN OF RESPONSIBILITY & TEMPLATE METHOD - 1 MIN 30 SEC

HAYTHAM

Sometimes standard WorkItems aren't enough. Some tasks need extra behaviour, but creating a new subclass for every possible combination is a nightmare. Decorator solves this. AuditedWork keeps a history you can edit. Escalated and ApprovedWork, on the other hand, tie directly into Chain of Responsibility.

We chain the receiving handlers together and just pass the request up the chain until someone handles it—moving one link at a time. handle acts as a Template Method here, outlining the skeleton of checking if the handler can process the request, and either running it or passing it along, with the FallbackHandler always handling a request, so nothing is left hanging.

COMMAND, MEMENTO & TEMPLATE METHOD - 1 MIN 30 SEC

RAYHAAN

Every operation in the system needs to be tracked and undoable. Command helps us achieve this by encapsulating every action as an object. CommandHistory is the invoker that runs the ConcreteCommands and manages the stack, with WorkItem and its decorators being the receivers.

To safely undo commands, we use Memento to restore the exact previous state without violating the object's encapsulation. Undo works repeatedly, newest first.

Before a command runs, it asks the WorkItem and every decorator layer for a snapshot. That covers who was assigned, how far along the chain we are, and who the current handler is. execute and undo in WorkCommand are Template Methods. There's a fixed skeleton of snapshot, perform and record, with perform being deferred to the relevant ConcreteCommand.

FACADE, ITERATOR & VISITOR - 1 MIN

KYLE

To keep things simple for the client, our Facade provides a unified, high-level interface to the complex subsystem classes, making it much easier to use.

To walk the workflow tree, we use a DepthFirstIterator, which accesses elements without exposing how they're stored.

And last but not least, Mark's favourite pattern, Visitor. We needed to generate progress reports and print the tree structure, but stuffing that code into the work classes would bloat them. WorkItem, WorkGroup and even the decorators accept a visitor, so ProgressReport counts the states, and TreePrinter draws the tree, tagging the decorated items.

INTRO TO SCENARIO 1 - 1 MIN

ORATILE *flips the switch. BLACKOUT. The lights cut. The projector screen is plain black.*

RAYHAAN

Wait, did the laptop just die?

HAYTHAM

Nah, I think it's loadshedding.

KYLE

Maybe it's another power outage?

TASHIL

Standard day in Hatfield, I guess...

ORATILE *flips the switch again. The lights return. The projector restarts: a bouncing 'NO SIGNAL' box drifts across the screen like a DVD screensaver.*

TASHIL

You know what, this actually gives me PTSD from the start of the quarter.

Test week. Rain pouring down. A substation decides to explode, and the whole of Hatfield just goes dark…

And so, we decided to turn our trauma into inspiration and model an Incident Response Workflow for a municipal power outage. From dispatching the crew to restoring the grid, Rayhaan will take you through it.

LIVE DEMO 1 - 6 MIN

RAYHAAN

No dialogue scripted.

INTRO TO SCENARIO 2 - 45 SEC

TASHIL

For our second scenario, we wanted to model something a little closer to home. We built a job application workflow for a tech company we highly admire: Tyto Insights.

We figured, what better way to impress Mark— I mean, demonstrate our generic engine—than by showing how it seamlessly handles Tyto's hiring pipeline? Who knows, maybe if we apply to Tyto, we'll go through this exact workflow.

ORATILE

And hopefully, our applications end up in the 'Approved' state.

LIVE DEMO 2 - 4 MIN

ORATILE

No dialogue scripted.

TESTING AND CI/CD - 1 MIN

HAYTHAM

With 14 patterns all interacting together, we needed to make sure everything works as it should.

We wrote comprehensive unit tests for each pattern using Google Test, integration tests for pattern combinations, and full end-to-end tests for both concrete scenarios. And we don't just run these locally. As part of our bonus marks, our CI/CD pipeline automatically creates a Docker container to run all these tests, followed by memory leak checks with Valgrind.

We also enforced a repository ruleset so that no pull request can be merged unless the pipeline passes.

DOCKER, DOXYGEN, GIT & TEAM CONTRIBUTION - 1 MIN

KYLE

You can think of our team as a linked list. Every node has its own attributes and roles.

To make sure our individual links formed a solid chain, we leaned heavily on Git. We each worked independently on our own feature branches and used Pull Requests to merge into ‘dev’. Once everything was stable in ‘dev’, we finally merged into ‘main’.

For bonus marks, we also used Docker to run the application inside a container.

Finally, we wrote all our own Doxygen comments to keep the codebase fully documented.

CONCLUSION - 15 SEC

TASHIL

And that’s TIVIDY: one generic engine, infinitely many scenarios, and five devs who made it possible. Thank you!

THE END